

Tuto Godot

0 – Prise en main

Voir les différentes parties de l'éditeurs

Voir les différents éléments manipulables (assets, nœuds, scripts)

- Différents nœuds : [2D](#), [3D](#), [Interface](#)
- Héritage des nœuds
- Comment ajouter un comportement
- Coordonnée d'un nœud

Voir rapidement la notion de frame / image.

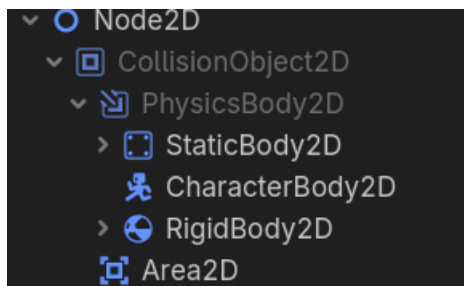
Quelques mots utiles :

- L'image d'un joueur ou d'un ennemi s'appel *Sprite* en anglais

1 – Ajout d'un joueur (Gestion des Nœuds)

Créer une nouvelle scène au bon endroit (nommé Player ou comme vous préférez)

Les nœuds qui bougent ont souvent besoin de collisions à un moment ou l'autre, pour cela on utilise ces nœuds :



Le *PhysicsBody2D* (et ses enfants) inclut la gravité, comme on fait un jeu vu du dessus sans gravité, on va utiliser un **Area2D**.

Chercher un nœud pour afficher l'image de notre joueur (pas besoin de l'animer pour l'instant).

Chercher un nœud pour lui ajouter une collision et lui donner une forme qui colle à notre joueur.

Importer la scène du joueur dans la scène principale.

2 – Mouvements (Découvertes des scripts)

Créer un script qui porte le même nom que la scène du joueur (au bon endroit).

Un script **extends** un type de node, par défaut on a la *Node* de base, mais il est préférable de le remplacer par le type de nœuds sur lequel on va l'attacher.

Ajout d'attributs :

- Structure de base : `@option_editeur var nom_variable : type = valeur`
- L'option éditeur et le type ne sont pas obligatoires
- L'option éditeur est utile pour ajouter un comportement à l'attribut

Essayer d'ajouter un attribut *speed* à notre joueur, qui va être sa vitesse à l'écran.

- Essayer l'option éditeur `@export` sur l'attribut *speed*.

Déplacer un nœud à l'écran

Dans un jeu, se déplacer c'est changer de position d'un frame à l'autre.

Dans Godot, les nœuds 2D ont tous une variable position, qu'on peut obtenir avec `position`. C'est un `Vector2`, contenant x et y, les coordonnées du nœud.

(On peut facilement modifier sa valeur en faisant `position = position + Vector2(x,y)`)

Pour savoir qu'on est passé au frame suivant, on utilise la méthode « `func _physics_process(delta: float) -> void:` » qui est appelé automatiquement par Godot à chaque frame.

Ajouter une méthode process comme celle-ci à notre joueur :

```
4  
5  func _physics_process(delta: float) -> void:  
6      print(delta)
```

Lancer le jeu et regarder la console de sortie.

Le delta qui apparait est le temps qui s'est écoulé depuis la dernière image, ça va nous permettre d'avoir le même mouvement qu'on soit à 30 fps ou à 240 fps.

Utiliser `position`, un `Vector2`, *speed* et *delta* dans `_physics_process()` pour que le joueur se déplace tout seul vers la droite quand on lance le jeu.

Gérer les entrées du joueur

En haut à gauche, le menu Projet->Paramètres du projet ouvre une fenêtre qui contient l'onglet Contrôles.

Définir 4 actions (haut, bas, gauche, droite) et y associer des touches

Dans le script du joueur, essayer d'écouter l'action haut avec `Input.is_action_pressed`. (en déplaçant le joueur ou en affichant un message)

Cette méthode prend un string en entrée qui est le nom de l'action dont on veut savoir s'il a été appuyé. Cette méthode est presque toujours utilisée dans un if. Dans Godot, on écrit un if comme ceci :

```
if(condition):  
>| print("oui")
```

Faire pareil pour les 3 autres actions, et modifier la position du nœud en fonction des actions du joueur.

3 – Ajout d'ennemis

Utilisez tout ce que vous avez appris sur le joueur pour faire une scène ennemie, qui va se déplacer automatiquement vers le haut.

Ajouter deux ennemis dans la scène principale, en dessous du joueur. Lancer et les regarder monter.

4 – Ajout de collisions (découverte des signaux)

Les signaux sont des événements appelés automatiquement par le moteur du jeu. On en a déjà vu plus haut avec `_physics_process()` qui est appelé automatiquement à chaque rendu d'image.

Dans la scène de votre joueur, cliqué sur le nœud tout en haut et aller dans signaux en haut à droite de l'éditeur.

Vous allez voir plusieurs signaux, chacun correspond à un type d'évènement de collision. Celui qui nous intéresse c'est le premier : `area_entered`.

Double clic gauche dessus vous propose de le lier à une méthode. Faites-le sélectionnez l'objet tout en haut et cliquez sur Connecter.

Vous devriez voir une nouvelle méthode dans votre script de joueur. Le prefixe `on` est une norme d'écriture pour dire que cette méthode sera appelée par un événement.

Essayez de `print()` un message dans la sortie quand notre joueur subit une collision. Nos ennemis ont déjà une zone de collision, la fonction sera donc appelée chez le joueur, même si elle n'est pas connectée chez l'ennemi.

Dans la scène de l'ennemi, connecter le même signal pour que cette fois l'ennemi disparaisse quand il touche le joueur.

Dans Godot, pour retirer un nœud de la scène on utilise la fonction `queue_free()`

5 – Ajout d'UI

Créer une nouvelle scène de type Control.

Ajouter 2 nœuds labels, un pour le nombre de PV restant et un pour le chrono

Ajouter un nœud Timer.

Créer un script pour notre UI. Utiliser le delta de la méthode `_process` pour compter le temps qui passe.

Utiliser l'option `@export` pour accéder aux labels depuis votre script.

Dans `_process`, mettre à jour le label du temps qui passe.

L'ajouter dans la scène principale.

Pour l'instant, nous allons appeler `_start()` directement depuis la méthode `_init()` du script de notre UI

6 – Ajout d'une boucle de jeu

Dans le script main, ajouter une méthode `debut_de_partie` et une `fin_de_partie`

Créer une UI de début. Utiliser un Panel pour faire un fond assombri et ajouter un bouton Commencer.

Le bouton a un signal qui permet de savoir quand on appuie dessus. Utiliser ce signal pour appeler la fonction `debut_de_partie()` de main.

Début de partie

Pour commencer la partie, on va faire apparaître un joueur au centre.

Pour faire apparaître un nœud, il faut le créer dans une variable en utilisant `Player.new()`, puis l'ajouter dans notre hiérarchie de nœud en utilisant `add_child(player)`

Ajouter un joueur à la scène quand on commence une partie.

Ajouter un timer, qui se lancera dès qu'on start et qui fera apparaître un ennemi tous les n seconde (utiliser le signal time_out)

Pour appeler le Timer, utilisez une variable de type **Timer** avec l'option **@export**.

Vous pouvez les faire apparaître comme vous voulez. Si vous hésitez, un bon moyen de commencer est de les faire apparaître sur un cercle autour du centre, en définissant leur position au hasard.

Obtenir un nombre aléatoire : **randf()**

Obtenir un angle : nombre * 2 * **PI** (Godot connaît **PI**)

Définir une position à partir d'un angle :

$x = \cos(\text{angle}) * \text{rayon}$

$y = \sin(\text{angle}) * \text{rayon}$

Vous vous souvenez la méthode pour les faire se déplacer vers le haut ? C'est plus dangereux de les faire aller vers le joueur ! Au moment de les créer, donnez-leur la position du joueur et utilisez la pour définir sa direction (une direction c'est la différence entre un point d'arrivée et un point de départ)

Sauvegardez la direction dans un variable et déplacez-vous l'ennemi dans cette direction.

Un ennemi qui sort de l'écran existe toujours, et au bout d'un moment l'ordinateur peut avoir du mal ! Notre écran n'est pas très grand, on peut considérer que si la position en x ou en y de l'ennemi est supérieur à 1000 ou inférieur à -1000, on le supprime.

Ajouter un signal au jour pour prévenir l'interface quand il prend des dégâts.

Ajouter un signal pour prévenir le main que le joueur est mort, pour arrêter la partie.

Si le joueur est touché 3 fois on arrête le jeu et on affiche le temps.

Ajouter un écran de fin, avec le temps affiché, et un bouton Recommencer qui appellera à nouveau debut_de_partie.

N'oubliez pas de tuer tous les ennemis à la fin d'une partie, sinon ils risquent d'être encore présent à la prochaine !

7 – Ajout d'animations

Remplacer le sprite par un animated sprite